

Lab1 Database Systems

Part 1:

Task1.1

1. Список суперключей /List of Superkeys:

EmpID
SSN
Email
Phone
EmpID, Name
SSN, Email, Phone

2. Кандидатные ключи /Candidate Keys:

EmpID
SSN
Email
Phone

3. Выбранный первичный ключ: {EmpID}

Почему именно он: это самый простой и удобный ID сотрудника. Он короткий, состоит только из цифр и выдается один раз насовсем. В отличие от почты или телефона, ID никогда не меняется, а в отличие от SSN не светит личные и секретные данные человека. В базах данных с такими айдишниками работать быстрее и безопаснее всего.

4. В таблице у всех трех сотрудников номера разные, так что формально в этой таблице телефон уникален и его можно считать кандидатным ключом.

Но в реальной компании телефон так себе ключ. Несколько сотрудников могут сидеть на одном рабочем номере, у них может быть общий городской номер отдела или человек просто поменяет симку. Поэтому на практике делать телефон ключом не стоит.

Отношение B: Регистрация на курсы /Relation B: Course Registration

1. Minimum Primary Key Attributes:

- StudentID, CourseCode, Section, Semester, Year

2.StudentID: Определяет конкретного студента, изучающего курс.

CourseCode: Определяет предмет/курс, который изучается.

Section: Разграничивает различные лекционные группы или время занятий по одному и тому же курсу в течение семестра.

Semester: Разграничивает записи по семестрам, поддерживая правило о том, что студенты могут повторно проходить курсы.

Year: Разграничивает записи по учебным годам.

3. Дополнительные кандидатные ключи:

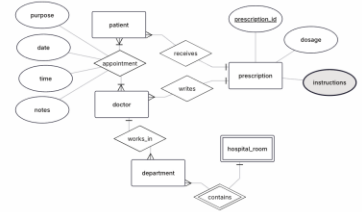
Их просто нет. Чтобы система не путалась, нам нужны все 5 полей вместе (StudentID, CourseCode, Section, Semester, Year). Если выбросить хотя бы одно поле — всё сломается. Например, без Semester нельзя будет понять, проходил ли студент предмет весной или осенью, а без Section в какую именно группу он ходит. Никакая комбинация из меньшего количества полей не сможет гарантировать уникальность.

Task1.2

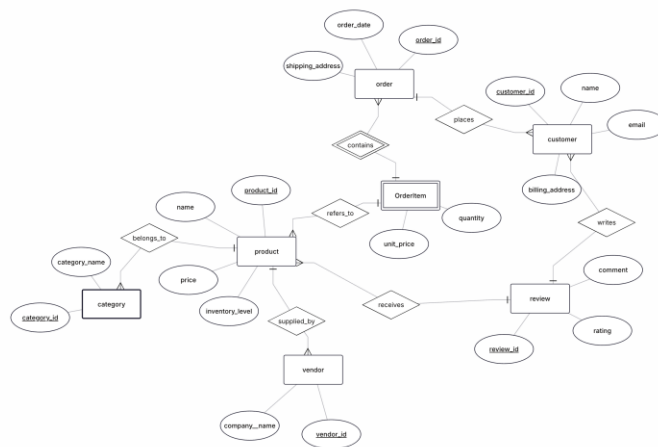
- **Student -> Professor:** Поле AdvisorID в таблице студентов указывает на ProfID преподавателя, который является его научным руководителем.
- **Course -> Department:** Поле DepartmentCode указывает, к какому факультету/кафедре (DeptCode) относится курс.
- **Department -> Professor:** Поле ChairID указывает на ProfID профессора, который заведует этой кафедрой.
- **Enrollment -> Student & Course:** Таблица записей содержит StudentID (какой студент) и CourseID (на какой курс), связывая их между собой.

Part 2:

Task2.1



task2.2



Part 4:

Task4.1:

1. Yes, there is. First of all, data is inconsistent. The data will repeat, if supervisor or project can change. That is 1NF violation where data should be consistent and not repeatable.
2. There is also 2NF violation there many PK's. SupervisorId, StudentId, ProjectId. We should decompose this one big table to many smaller tables such as

Supervisors, Students, Projects where PK's will really be PK to make the data reusable.

3. Finally, we can make another transitive table where all of the data will be connected each other, for example we will call it 'StudentProjects'. There will be ForeignKeys such as StudentID, SupervisorID, ProjectID and Role, HoursWorked, Start, End dates. That will be final normalized table.

Task4.2:

1. Primary Key of the Table

Primary Key: {StudentID, CourseID}

Why: Since a student can enroll in multiple courses and a course has many students, we need both StudentID (who is taking it) and CourseID (which course they are taking) together to uniquely identify each row in the schedule.

2. Functional Dependencies

Based on the business rules, we have the following dependencies:

StudentID StudentMajor (*Each student has one specific major*)

CourseID CourseName, InstructorID, TimeSlot, Room (*A course section has a set name, teacher, time, and room*)

InstructorID InstructorName (*An instructor ID determines their name*)

Room Building (*Rooms are unique across campus and determine the building*)

3. BCNF Analysis

Is it in BCNF? No, it isn't (it actually breaks 2NF and 3NF as well).

Reason: For a table to be in BCNF, every determinant (the left side of an arrow) must be a candidate key. Here, attributes like StudentID, CourseID, InstructorID, and Room determine other columns on their own, but none of them are full keys by themselves.

4. BCNF Decomposition

To fix the redundancy and update anomalies, we break the table down into 5 smaller tables:

Student (StudentID [PK], StudentMajor)

Course (CourseID [PK], CourseName, InstructorID, TimeSlot, Room)

Instructor (InstructorID [PK], InstructorName)

RoomLocation (Room [PK], Building)

Enrollment (StudentID, CourseID) — *Composite PK: {StudentID, CourseID}*

5. Loss of Information Analysis

Lossless Join: There is no loss of information. This decomposition is completely lossless because all relationships are preserved through foreign keys. We can easily bring back the original CourseSchedule table at any point using standard SQL JOIN operations on the keys without creating fake rows or losing data.

Part 5:

